



SMART CITY MULTIMODAL MOBILITY

Case File: The STN_003 "Hardware Failure"

Tools: Python, Pandas, NumPy, Matplotlib/Seaborn

The Case

The city's Transit Authority has filed an incident report:

Official Report: *"On Day 15, Subway Station STN_003 experienced a hardware failure between 14:00 and 20:00, resulting in dropped turnstile logs. No further investigation is required."*

You are not convinced. The Transit Authority never cross-referenced their own sensor logs against the city's independent weather feed or the scooter-share network's GPS trails. You have three raw, messy datasets. Your job as a data analyst is to clean them, join the dots, and prove — or disprove — the hardware failure theory.

You will be working with:

File	Description
scooter_rides.csv	~5,000 individual e-scooter trip records across the city
subway_turnstile.csv	~2,000 hourly turnstile entry/exit counts across 10 stations
city_heartbeat.json	30 days of nested city-wide weather and civic sensor data

Your deliverable: A short written verdict (3–5 sentences) backed by a multi-panel visualization, submitted alongside your annotated notebook.

Ground Rules

- Do not delete rows just because they look messy — figure out *why* they're messy first.
- Document every assumption you make (e.g., "I assumed epoch timestamps are in UTC seconds").
- You are allowed to create intermediate/helper columns. Keep your notebook readable — a mystery is easier to solve when your workings are visible.

Phase 1 — Cleaning & Standardization ("The Grunt Work")

Nothing in this case can be trusted until it's clean. Before you can look for a flood, you need data you can actually filter, sum, and compare.

1.1 String Operations

Your station_id column is a mess of casing and padding — 'STN_003 ', ' stn_003', 'Stn_003' are all the same station, but Pandas doesn't know that yet.

Line of Action:

- Inspect `subway_turnstile.csv['station_id'].unique()` to see the full scope of the inconsistency.
- Use `.str.strip()` and `.str.upper()` (or `.str.lower()`) to collapse all variants into a single canonical form.
- Repeat the same instinct for `start_lat` / `start_lng` in `scooter_rides.csv` — some values are clean floats, others padded strings like `' -74.0060 '`, and others carry corrupted suffixes like `'40.7128° N'`.
- Strip whitespace, strip the `°`, `N`, `W`, and `%` characters where relevant, before attempting any numeric conversion.

✓ **Checkpoint:** After cleaning, `station_id.nunique()` should return exactly 10, and every value in `start_lat` / `start_lng` should be convertible to a float.

1.2 Type Casting

Several columns are storing numbers as text, and some of that text isn't even numeric.

Line of Action:

- `battery_level` mixes integers, `'85%'`, whitespace-padded numbers, and outright non-numeric strings (`'LOW'`, `'N/A'`). Strip any non-digit characters, then cast to a numeric type using `pd.to_numeric(..., errors='coerce')` so unsalvageable values become NaN instead of crashing your pipeline.
- entries and exits in the turnstile data contain the string `'DEVICE_ERROR'` mixed in with legitimate integer counts. Apply the same `errors='coerce'` pattern.
- `user_rating` is 10% blank and 5% written as text (`'five star'`, `'1.0'`). Decide on — and document — a strategy: do you map text values to numbers, or treat them as missing?
- `start_time` / `end_time` arrive in **three different formats**: ISO 8601 strings, US-style MM/DD/YYYY HH:MM AM/PM strings, and raw Unix epoch integers. A single `pd.to_datetime()` call will not handle all three at once — you'll need to detect the format per-row (hint: check if the value is numeric first) before parsing, or parse in batches by format.

✓ **Checkpoint:** `df.dtypes` should show proper `datetime64`, `float64`, and `int64` types where appropriate — no more object columns hiding numbers.

1.3 Handling Anomalies

Turnstiles cannot log a negative number of people. Yet 5% of your entries/exits values are negative.

Line of Action:

- Quantify the problem first: what percentage of rows have negative values? Is it random noise, or concentrated somewhere?
- Decide on a defensible handling strategy (e.g., take the absolute value if you believe it's a sign-flip bug, or treat as missing if you believe it's a corrupted read) — and justify your choice in a markdown cell.
- Do **not** silently drop these rows without checking whether they cluster around a particular station, day, or hour — that clustering might itself be a clue.

Phase 2 — Aggregation & Group By ("Finding the Signal")

With clean data in hand, it's time to zoom out from individual rows to daily and hourly patterns.

Line of Action:

- Use `.groupby()` on your cleaned subway_turnstile data to calculate:
 - Daily total entries/exits **per station**
 - Average hourly turnstile flow **per station**
- Use `.groupby()` on your cleaned scooter_rides data to calculate daily ride counts and average battery levels city-wide.
- **The Bounding Box:** STN_003 sits at approximately (40.7128, -74.0060). Define a lat/lng bounding box (e.g., ± 0.006 degrees in each direction) around this point, and filter scooter_rides down to only the rides whose start_lat/start_lng fall inside it. This isolates scooter activity in STN_003's immediate neighborhood — the population you'll need for Phase 4.

✓ **Checkpoint:** How many scooter rides fall inside your STN_003 bounding box? Sanity-check this number against the city-wide total — does it look like a plausible neighborhood share?

Phase 3 — Advanced Reshaping ("The Pivot Table")

Aggregates are useful, but a well-built pivot table can make an outlier visually undeniable in seconds.

Line of Action:

- Filter your cleaned turnstile data down to **Day 15 only**.
- Build a `.pivot_table()` with hour as the index, station_id as the columns, and entries (or exits) as the values, aggregated with mean or sum.
- Scan the resulting table (or better, apply a conditional style / heatmap) across all 10 stations for Day 15. One column should visibly collapse between certain hours while its neighbors stay steady.

✓ **Checkpoint:** Can you state, in plain language, exactly which station and which hour range breaks the pattern — using only the pivot table as evidence?

Phase 4 — Multimodal Visualization ("The Reveal")

This is where you present your case. A single chart type won't do it justice — you need to overlay three independent signals and let the city tell you what actually happened.

Line of Action:

- Keep it simple: build **3 stacked panels** that all share the same x-axis (hour of day, 0–23), using `plt.subplots(3, 1, sharex=True)`. One signal per panel:
 - **Panel 1 — Hourly subway entries for STN_003** (line or bar) — from your Phase 3 pivot table.
 - **Panel 2 — Scooter rides per hour near STN_003** (bar) — count rides starting each hour on Day 15, from your Phase 2 bounding-box subset.

- **Panel 3 — Precipitation on Day 15** (line or bar) — from `city_heartbeat.json` (remember: the date keys are inconsistently formatted, so standardize them before looking up "Day 15").
- Give each panel a short title and a y-axis label. Only the bottom panel needs an x-axis label ("Hour of Day").
- Use `axvspan(14, 20, alpha=0.2)` on each panel to shade the disputed 14:00–20:00 window, so the reader can scan straight down and see how all three signals behave at the same moment.

Final Verdict

In a markdown cell (or short paragraph), answer directly:

Question: *Did STN_003 suffer a hardware failure on Day 15 — or did something else happen?*

Support your answer with specific numbers pulled from your own analysis: the size of the entries drop, the precipitation reading at the time, and what (if anything) the scooter data around the station did during that same window.

Submission Checklist

- ☐ Notebook runs top-to-bottom without errors
- ☐ All three raw files loaded and cleaned with documented assumptions
- ☐ Bounding-box filter applied and justified
- ☐ Day-15 pivot table built and interpreted
- ☐ Final multi-panel chart with clear annotations
- ☐ Written verdict, backed by numbers, in your own words